

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

UNIVERSITY OF SCIENCE

INSTITUTION OF INFORMATION TECHNOLOGY

Fitness Mobile Application

Final Project Report

Course: **CS426 - Mobile Device Application Development**

Class: **24A01**

Students:

Nguyễn Gia Hào (24125008)

Cao Quảng Hưng (24125010)

Huỳnh Chí Tôn (24125020)

Bùi Nguyễn Đức Trọng (24125021)

Lecturers:

Trần Minh Triết

Teaching Assistants:

Đỗ Trọng Lễ

Huỳnh Viết Thám



September 2026

Contents

1	Project Overview	1
1.1	Problem	1
1.2	Project Goal	1
1.3	Project Scope	2
1.4	Repository Snapshot	2
2	Target Users and Main Use Cases	3
2.1	Target Users	3
2.2	Profile and Recommendation Use Case	3
2.3	Typical End-to-End Flow	4
2.4	Alternative Flows	4
3	Implemented Features	4
3.1	Training Home	4
3.2	Exercise Library	5
3.3	Plan Recommendation	5
3.4	Plan Details and Editing	6
3.5	Custom Workout Plans	6
3.6	Guided Workout Player	6
3.7	Progress, Streaks, and Achievements	7
3.8	Workout History and Weight Tracking	7
3.9	Settings, Reminders, and Voice	8
3.10	AI Fitness Assistant	8
3.11	Feature Coverage Summary	9
4	Application Architecture	10
4.1	Architectural Style	10
4.1.1	UI Layer	10
4.1.2	Domain Layer	11
4.1.3	Data Layer	11

4.2	Dependency Construction	11
4.3	Navigation	11
4.4	Local Data Model	12
4.5	Database Creation and Migration	12
4.6	Concurrency and Reactive State	13
4.7	AI Request Flow	13
4.8	Security and Privacy	14
5	Technologies Used	14
6	Setup and Installation	15
6.1	Requirements	15
6.2	Clone and Open	15
6.3	Configure Local Properties	16
6.4	Build from the Command Line	16
6.5	Runtime Permissions	16
6.6	Important Branch Setup	17
6.7	Verification Commands	17
7	Work Division	17
7.1	Shared Responsibilities	17
8	Evaluation and Self-Assessment	18
8.1	Assessment Method	18
8.2	What Was Completed	19
8.3	Strengths	19
8.4	Known Gaps and Risks	20
8.4.1	Branch and Database Integration	20
8.4.2	Automated Tests	20
8.4.3	Static and Partially Connected UI	20
8.4.4	AI Scope and Safety	20
8.4.5	Platform and Quality Checks	21

8.5 Overall Self-Assessment	21
8.6 Recommended Next Steps	22
9 Conclusion	22
10 AI Usage Statement	23
References	24

Abstract

HomeWorkout is an Android application that helps people plan and complete structured exercise sessions without depending on a gym coach or a continuous internet connection. The application combines a local exercise library, profile-based plan recommendation, editable multi-day workout plans, guided workout playback, progress tracking, reminders, achievements, weight monitoring, and an experimental AI fitness assistant.

The project is implemented in Kotlin with Jetpack Compose. It follows a three-layer structure made of UI, domain, and data layers. ViewModels expose observable state, use cases contain focused application rules, repositories separate domain models from storage and network details, and Room provides the local source of truth. The bundled dataset contains 771 usable exercises and the recommendation catalog contains 16 plan templates. Most training functions continue to work offline after the first installation. The chatbot is the main network-dependent function and calls the Groq API through OkHttp.

This report explains the problem addressed by the application, its target users, the implemented user flows, architecture, database design, technology choices, setup process, and team responsibilities. It also evaluates the current implementation honestly. The main training, plan editing, settings, reminder, streak, and achievement flows are implemented. Weight and database-backed history work are available on the main development line, while the chatbot is implemented on a separate branch that has not yet been safely integrated with the latest database migration. The current prototype builds successfully, but automated test coverage and production security still require further work.

1 Project Overview

1.1 Problem

People who exercise at home often face three practical problems. First, they may not know which exercises match their goal, experience, available time, and equipment. Second, a fixed plan becomes inconvenient when the user wants to replace an exercise, change repetitions, or create a personal schedule. Third, motivation is difficult to maintain when progress is not visible and the application provides no reminders or feedback.

Many fitness applications solve only one part of this process. A video library can demonstrate movements but may not organize them into a plan. A workout timer can guide a session but may not explain the exercises. A generic chatbot can answer questions but may not preserve the user's training records. HomeWorkout was designed as one connected flow: choose a suitable plan, inspect and edit it, complete one workout day at a time, and review progress afterward.

1.2 Project Goal

The goal is to provide a usable mobile workout companion for general fitness. The application should reduce the effort required to start a routine while still allowing personal control. Its main

objectives are:

- recommend a plan from a structured fitness profile;
- provide searchable exercise information with images, animated demonstrations, muscles, equipment, and instructions;
- allow users to add, replace, reorder, or remove exercises and create custom multi-day plans;
- guide a workout through preparation, exercise, rest, and completion phases;
- save completed sessions and calculate weekly goals, streaks, achievements, history, and weight indicators;
- persist settings and schedule daily workout reminders;
- provide a small AI assistant for fitness questions while keeping conversation history locally.

1.3 Project Scope

HomeWorkout is a single-user Android prototype. The user account is created locally during database seeding, and there is no sign-in server or cloud synchronization. Exercise data, plan templates, settings, workout sessions, badges, weight records, and chat history are stored on the device. Network access is used for remote exercise media and the AI assistant.

The application focuses on planning, guidance, and personal tracking. It does not diagnose injuries, replace medical advice, verify exercise form through the camera, process payments, connect to wearable devices, or provide a social network. These boundaries keep the project feasible for a university mobile-development course and make the implemented behavior easier to verify.

1.4 Repository Snapshot

The inspected repository contains 166 Kotlin source files, 23 UI feature files with Compose content, 34 focused use-case files, 18 Room entity classes on the chatbot branch, 771 filtered exercises, 16 plan templates, and 12 profile questions. The application uses one Gradle module named `app`. Three persistent bottom-level destinations organize the product into Training, Report, and Settings, while feature screens are opened through the Navigation Compose graph.

Development is distributed across several Git branches. The current `chatbot` branch contains the AI assistant, while `origin/main` contains newer weight and workout-history work. This report covers the completed work found across the repository and identifies the remaining integration work in the self-assessment section.

2 Target Users and Main Use Cases

2.1 Target Users

The primary target is a beginner or intermediate user who wants to exercise at home and needs more structure than a list of videos. This user may have limited equipment, limited time, or uncertainty about how to arrange exercises into a weekly routine. The application also supports experienced users who prefer to modify an existing plan or build a custom one.

Table 1: Target user groups

User group	Main need	Application support
New exerciser	A safe and understandable starting point	Beginner filtering, profile questions, recommended plans, exercise instructions, and guided timers
Busy user	A plan that fits a limited schedule	Plan matching by days per week and session duration, plus short or bodyweight plans
Home trainee	Exercises compatible with available equipment	Equipment-aware plan generation and exercise filters
Returning user	Motivation and visible continuity	Weekly goal, streak, achievements, reminders, history, and weight trends
Independent trainee	Control over the routine	Custom plans and add, replace, delete, reorder, or repetition editing
User needing quick guidance	Short answers about exercise and recovery	Floating AI fitness assistant with saved conversations

2.2 Profile and Recommendation Use Case

The onboarding flow collects the user's main goal, experience level, available days, expected session duration, equipment, preferred training areas, and recorded limitations. The supported goals are general fitness, muscle building, fat loss, mobility, and a focused body area. Experience is divided into beginner, intermediate, and expert levels.

The recommendation logic first applies hard constraints. For example, a beginner receives beginner exercises, selected exercises must match available equipment, and explicit exclusions should not be relaxed. It then ranks templates using goal, level, weekly schedule, duration, equipment coverage, and focus preferences. A deterministic tie-break keeps the result stable for the same inputs. The selected catalog template is materialized into Room as a normal editable workout plan.

2.3 Typical End-to-End Flow

A normal user journey is:

1. Open Training and complete the fitness-profile questions.
2. Review the recommended plans and select one.
3. Open a plan to see its days, exercises, targets, images, and level.
4. Inspect an exercise or edit the plan when equipment or preference changes.
5. Start the next planned day. The app creates a workout-session record.
6. Follow the preparation, active exercise, and rest phases. The user can pause, skip, move backward, or finish the current exercise.
7. Complete or abandon the session. Completion updates progress and triggers badge evaluation.
8. Review the weekly goal, streak, achievements, history, or weight dashboard.
9. Adjust timers, audio, units, voice, and reminder settings when needed.

2.4 Alternative Flows

The user is not forced to accept the recommendation. They can browse workouts by category, create a custom multi-day plan, choose exercises from the full library, or modify a system plan. If a session was abandoned, starting the same plan resolves to the same day. If the latest session was completed, the next day is selected, and the sequence wraps to day one after the last day.

The AI assistant is available as a draggable overlay above the navigation graph. A user can create multiple conversations, ask a question in Vietnamese or English, close the panel without losing history, and delete sessions that are no longer needed. The model is instructed to remain within fitness, recovery, exercise, and basic nutrition topics.

3 Implemented Features

3.1 Training Home

The Training screen is the main starting point. It loads workout plans and progress through `HomeViewModel`. The screen presents the current streak, coach tips, recommended challenges, a weekly-goal card, workout categories, existing plans, and a route to custom plan creation. Data is exposed as `StateFlow`, so changes in Room can update the visible list without manual refresh logic.

The weekly-goal card compares the configured target with completed workout days in the active week. It marks today, completed days, and the number of days still required. A separate Edit Goal screen changes the weekly target and the first day of the week.

3.2 Exercise Library

The local JSON asset originally represents a larger exercise source, but the current seed contains 771 usable records after entries with unavailable media were removed. Every exercise can contain:

- a title and category;
- a difficulty level and force type;
- required equipment;
- primary and secondary muscles;
- ordered instruction steps;
- one GIF demonstration and optional images.

Users can search by text and filter by category, level, and equipment. The same browser is reused for adding exercises to a plan and replacing an existing exercise. The Exercise Information screen shows the movement media, muscles, equipment, level, and numbered instructions. Coil loads normal images and animated GIF files; a placeholder is displayed when media cannot be loaded.

3.3 Plan Recommendation

The application ships with 16 catalog plans. They cover general fitness, muscle building, fat loss, body-area focus, mobility, and travel-oriented bodyweight training. Plans range from beginner to expert or use an adaptive level for focused programs. Examples include Beginner Full Body Starter, Intermediate Upper Lower Builder, Expert Conditioning, Back and Posture Focus, and Travel Bodyweight Plan.

The recommendation algorithm is rule-based instead of AI-generated. This choice makes the result repeatable and prevents the model from inventing exercise identifiers. Template scoring gives the highest weight to goal match, followed by level, weekly availability, duration, equipment, and focus. For each required slot, exercise selection progressively relaxes muscle and force preferences when the candidate pool is too small, but it keeps safety and equipment constraints.

When the user accepts a recommendation, the selected template is converted into a normal Room plan with days and exercise targets. The saved fitness profile records the catalog identifier and catalog version so the origin of the recommendation remains available.

3.4 Plan Details and Editing

The Plan Detail screen displays plan metadata and groups exercises by workout day. A preview use case identifies which day will start next without opening a session. This keeps display logic separate from the operation that writes the session record.

Editing supports the following operations:

- add one or more exercises to a selected day;
- replace a plan exercise with an exercise selected from the browser;
- update a repetition target;
- delete an exercise from a day;
- reorder exercises through drag-and-drop.

Each operation is represented by a focused use case and coordinated by a dedicated plan-editing ViewModel. The UI does not call a DAO directly. This makes the same rules reusable and prevents Compose code from depending on Room entities.

3.5 Custom Workout Plans

Users can create plans instead of modifying only the bundled plans. The custom-plan builder collects a title, optional description, category, level, and one or more workout days. Every non-empty day contains selected exercises and their targets. The repository writes the plan, days, and plan exercises as one domain operation. Custom plans appear in a separate list and can be opened through the same detail, editing, and player flows as system plans.

Deletion is limited to custom plans. This protects the bundled catalog content while allowing a user to manage personally created routines.

3.6 Guided Workout Player

Starting a plan creates a session associated with one plan day. The player uses four phases:

1. **Preparation:** show the next exercise and count down before work starts.
2. **Exercise:** display the GIF, exercise title, repetitions or remaining time, and transport controls.
3. **Rest:** show the next exercise, count down, allow an additional 20 seconds, or skip.
4. **Completed:** report the number of completed exercises and offer exit or restart actions.

The user can pause a timed exercise, return to the previous item, skip forward, open exercise information, finish a repetition-based item manually, or quit the session. A completed session receives an end timestamp and measured duration. A quit session is marked abandoned. Restarting after completion creates a new session for the same plan day rather than advancing automatically.

At session creation, the repository snapshots timer, music, sound, coach-video, and voice preferences. This protects session history from later settings changes. However, the current player UI still uses fixed preparation and rest constants rather than the saved values. This is listed as incomplete integration rather than described as finished behavior.

3.7 Progress, Streaks, and Achievements

Completed session timestamps are the source for weekly-goal and streak calculations. The streak use case groups completed workouts by calendar day, calculates consecutive days, and exposes current and personal-best values. Abandoned and in-progress sessions do not count.

The achievement system contains ten badges across four metrics:

- total completed sessions;
- best streak length;
- total completed workout duration;
- number of completed plans.

Thresholds include the first completed workout, five, ten, twenty-five, and one hundred workouts; three, seven, and thirty-day streaks; ten total workout hours; and one fully completed plan. Badge progress is calculated from live session totals. Newly eligible badges are inserted once and can trigger an unlock dialog after a workout. The Report and Achievements screens show unlocked state, tier, and progress.

3.8 Workout History and Weight Tracking

The `origin/main` development line replaces the earlier static history sample with database-backed history. It adds session snapshots for the plan title, plan cover, day number, and day title so an old record remains meaningful after the user edits or deletes a plan. History displays only completed sessions with an end time, marks workout days in a calendar, and calculates a weekly summary of workouts, duration, and calories when available.

The same development line adds a weight dashboard. Weight is stored in kilograms with a height snapshot. Domain logic calculates the current, heaviest, lightest, and seven-day average weight, seven-day change, BMI, BMI category, and up to seven recent chart records. The report card

links to a dedicated screen where the user can record weight and update height.

These functions are implemented and tested on `origin/main`, but the current chatbot branch predates them. They must be merged before one APK contains both the newer history and the AI assistant.

3.9 Settings, Reminders, and Voice

Settings are divided into workout, general, and voice sections. Stored preferences include the weekly goal, first day of week, metric or imperial units, preparation and rest timers, music and sound toggles and volumes, keep-screen-on behavior, reminder status and time, and the preferred text-to-speech voice.

Daily reminders use `AlarmManager`. When the alarm fires, a receiver creates a high-priority notification and schedules the next occurrence. A boot receiver restores the reminder because Android removes scheduled alarms after device restart. Android 13 and later require notification permission. Exact alarms and aggressive vendor battery management may require additional device settings, so the UI provides permission and diagnostic guidance.

The TTS service lists voices reported by the installed Android speech engine, prefers offline voices, supports preview, and falls back to pitch shaping when distinct male and female voices are unavailable. Keep-screen-on is connected to the activity window. Music and sound values are persisted, but the repository contains no complete audio playback engine, so these controls currently represent saved configuration rather than a finished media feature.

3.10 AI Fitness Assistant

The chatbot branch adds a draggable bubble that is mounted above the whole navigation graph. Tapping it opens a compact panel with a session list or an active message thread. Users can create, open, and delete conversations. Markdown replies support headings, emphasis, lists, links, and code-style formatting without using a `WebView`.

Each message is saved locally before the network request. The Groq client sends a system instruction, a rolling context summary, and the newest user message to the OpenAI-compatible chat-completions endpoint. The selected `openai/gpt-oss-20b` model is requested to produce strict JSON containing two fields: the visible reply and an updated summary. Storing the summary avoids resending the full transcript on every turn. Groq structured output provides schema-constrained JSON for supported models [1].

Failures are caught in the repository. A friendly assistant message is stored in the conversation, while the technical error is written to Logcat. Coroutine cancellation is rethrown so closing the screen can cancel active work correctly.

The current assistant is a general fitness chatbot. It does not yet receive the active exercise, plan,

fitness profile, or recent history automatically. The API key is read from `local.properties` and compiled into `BuildConfig`; this is convenient for a course prototype but is not secure for public distribution because an APK can be inspected. A production design should place the provider key behind a controlled backend or use a mobile-focused authorization mechanism.

3.11 Feature Coverage Summary

Table 2: Feature implementation status

Area	Status	Evidence in repository
Exercise library and detail	Completed	Room entities and DAOs, 771 seeded exercises, search filters, images, GIFs, muscles, and instructions
Recommendation and onboarding	Completed	12 profile questions, 16 plan templates, deterministic scoring and materialization
Plan editing and custom plans	Completed	Add, replace, reorder, delete, repetition editing, create and delete custom plans
Workout player and sessions	Mostly completed	Four phases, session completion and abandonment; configured timer values are not yet consumed by the player
Goals, streaks, badges	Completed	Database-derived weekly progress, streak calculation, ten-badge catalog, unlock workflow
Settings and reminders	Mostly completed	Persistence, exact alarms, boot restore, notification and TTS; music playback is not implemented
History and weight	Completed on main branch	Database-backed history, migration test, weight records, chart, BMI and dedicated screen
AI chatbot	Prototype completed on chatbot branch	Global overlay, Room history, Groq call, rolling summary, structured reply and fallback handling
Integrated release	Not completed	Main and chatbot independently use database version 6 with incompatible schemas

4 Application Architecture

4.1 Architectural Style

HomeWorkout uses a layered architecture with a domain layer between the UI and data layers. This structure follows the Android recommendation to separate UI state, business rules, and data sources [2]. It also uses unidirectional data flow: a screen sends user actions to a ViewModel, the ViewModel invokes use cases, and observable state flows back to Compose.

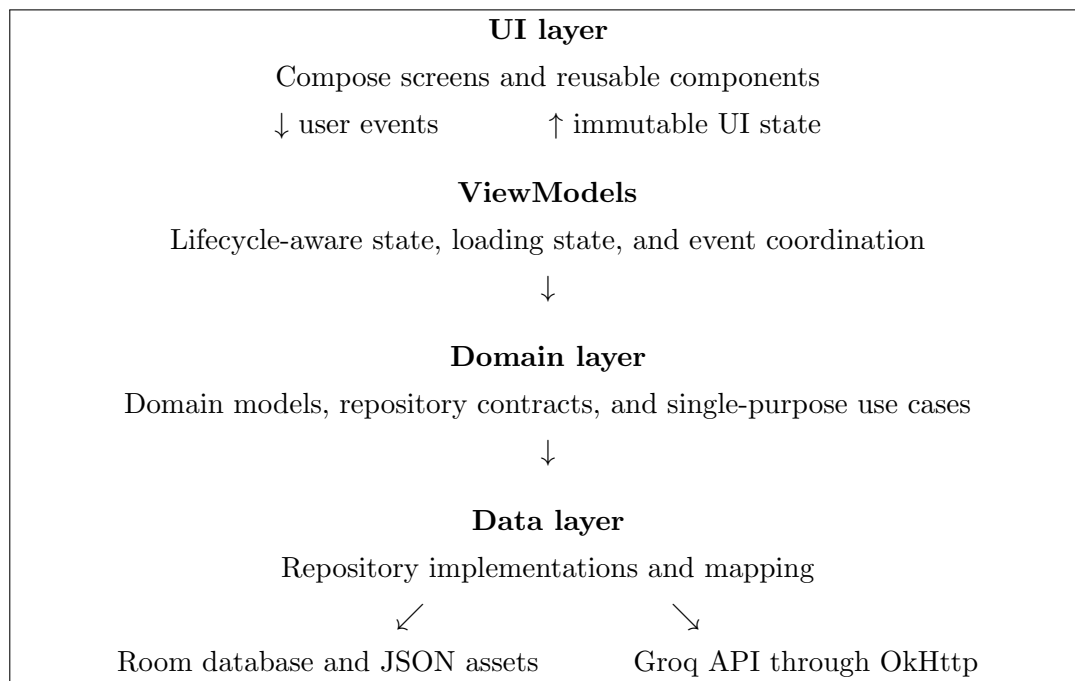


Figure 1: High-level dependency and data flow

4.1.1 UI Layer

The UI layer is built with Jetpack Compose. Each main feature has a screen and, where state or business operations are required, a ViewModel. ViewModels expose `StateFlow` values or sealed UI states such as Loading, Success, Empty, and Error. Screens collect these flows with lifecycle-aware Compose APIs.

Reusable components include buttons, cards, exercise rows, thumbnails, settings rows, dialogs, wheel pickers, stat tiles, badge visuals, and theme-based plan images. Colors, typography, shapes, and gradients are centralized under `ui/theme`. This reduces repeated styling and keeps screen behavior separate from presentation primitives.

4.1.2 Domain Layer

The domain layer contains pure Kotlin models and repository interfaces. It has no Room annotations or Compose types. Use cases represent one action or calculation, such as searching exercises, recommending a plan, starting a session, resolving the next workout day, updating repetitions, computing a streak, or sending a chat message.

This boundary gives the project a clear vocabulary. A UI screen works with **Exercise**, **FitnessProfile**, or **WorkoutPlanDetail**, not with database rows. It also prevents SQL details from spreading into ViewModels.

4.1.3 Data Layer

Repository implementations are the boundary between domain objects and concrete sources. Most repositories read and write Room through DAOs. The catalog repository reads a bundled JSON file. The chat repository combines Room persistence with a remote Groq client. Mapping functions convert Room entities and DAO projection rows into domain models.

This arrangement is consistent with Android's guidance that repositories should expose data while UI components should not access a database or network source directly [3].

4.2 Dependency Construction

The project uses manual dependency injection rather than Hilt or Koin. The custom **Application** class creates the Room database, repositories, services, and use cases with lazy properties. Navigation destinations build ViewModels through **viewModelFactory** and pass only the required dependencies.

Manual construction is understandable for a single-module course project and avoids another framework. The trade-off is that the **Application** class grows as features are added. A larger production version would benefit from dependency modules and test replacements.

4.3 Navigation

Navigation Compose manages all destinations in a single activity. The bottom navigation contains Training, Report, and Settings. Detail flows use typed route helpers with plan, plan-day, plan-exercise, exercise, or category identifiers. The bottom bar is hidden on pushed screens so the user can focus on editing or completing a workout.

The AI assistant is deliberately not a navigation destination. **HomeWorkoutApp** places the navigation host and **ChatOverlay** inside one top-level box. This allows the draggable bubble to remain available above every feature screen and lets its ViewModel survive normal destination changes.

4.4 Local Data Model

Room is used because it provides compile-time query checks, entity-based schema definition, migrations, and Flow integration [4]. On the chatbot branch, the database registers 18 entities. They can be grouped as follows:

Table 3: Persistent data groups

Data group	Main tables	Purpose
User and preferences	users, user settings, fitness profiles	Local identity, onboarding answers, weekly goal, timers, units, reminders, audio and voice choices
Exercise catalog	equipment types, muscles, exercises, exercise muscles, instruction steps, exercise images	Searchable movement library and detailed guidance
Workout plans	workout plans, plan days, plan exercises	System and custom multi-day routines with ordered targets
Workout activity	workout sessions, session exercises	Session status, timestamps, duration, calories, settings snapshot, and performed-exercise snapshot
Progress	weight logs, user badges	Weight history and one-time achievement unlocks
Chat	chat sessions, chat messages	Conversation titles, rolling summaries, roles, timestamps, and message content

Foreign keys connect child rows to users, plans, days, sessions, and exercises. Cascade deletion is used where child data has no meaning without its parent. Order indexes preserve exercise and instruction order. Enums are stored by name through Room type conversion.

4.5 Database Creation and Migration

On first creation, a callback starts the database seeder on an application-level IO coroutine. The seeder creates a local default user and settings, imports exercises and their related records, and materializes starter plans from the catalog.

Schema changes are versioned. Version 5 added persisted badges. On the main branch, version 6 adds four nullable plan/day snapshot columns to workout sessions and backfills them where possible. The chatbot branch also declares version 6 but uses it to add chat tables. This creates

two incompatible schemas with the same version number. Room detects the different identity hashes and refuses to open an existing database. The correct integration is:

1. use the main branch version 5-to-6 migration for history snapshots;
2. merge the chat entities and DAO;
3. increase the database version to 7;
4. add a version 6-to-7 migration that creates both chat tables and indexes;
5. run migration tests from versions 4, 5, and 6.

4.6 Concurrency and Reactive State

Kotlin coroutines execute database and network work without blocking the UI thread. ViewModels launch actions in `viewModelScope`. Room queries return `Flow`; ViewModels convert them to `StateFlow` with a five-second while-subscribed policy where appropriate. The chat HTTP call explicitly runs on `Dispatchers.IO`. Cancellation exceptions are preserved instead of being converted into normal error messages.

The workout timer is an in-memory Compose state machine driven by a one-second delay loop. This is simple and responsive while the screen stays active. Fine-grained progress inside the current day is not persisted, so leaving an unfinished workout restarts that day.

4.7 AI Request Flow

One chat turn follows this sequence:

1. The ViewModel rejects blank input and prevents concurrent sends.
2. The repository writes the user message to Room.
3. The first message becomes a short session title.
4. The repository reads the current rolling summary.
5. The Groq client sends the summary and new message with a fitness-scoped system instruction.
6. The model returns strict JSON with `reply` and `updatedContext`.
7. Both the assistant reply and summary are persisted.
8. Room Flow updates the visible message thread.

This design uses one model request per user turn and limits context growth. It does not yet ground answers in the app's exercise database, and safety handling is mainly prompt-based. A stronger version should add local risk checks, provide relevant exercise and plan context, and avoid allowing AI output to write directly to the plan database.

4.8 Security and Privacy

The core app keeps personal data locally and does not require an online account. Android backup remains enabled, so data may be included according to the configured backup rules. Chat messages are sent to a third-party AI service. Users should be informed not to enter sensitive medical or identity information.

The Groq token is excluded from Git because it is stored in `local.properties`, but the build script places it in `BuildConfig`. Excluding a source file protects the repository; it does not protect the final APK. For a public release, the API call should pass through a backend that keeps the provider secret, applies rate limits, validates requests, and allows token rotation.

5 Technologies Used

Table 4: Main technologies and their roles

Technology	Version or configuration	Role
Kotlin	2.2.10	Main programming language for UI, domain logic, repositories, services, and tests
Android Gradle Plugin	9.2.1	Android build, packaging, resource processing, lint, and variants
Gradle Wrapper	9.4.1	Reproducible command-line build environment
Android SDK	compile 37, target 36, minimum 24	Compilation API, declared behavior target, and device compatibility boundary
Jetpack Compose	BOM 2026.02.01	Declarative UI, state-driven rendering, layouts, animations, and Material components
Material 3	1.4.0	Design system, controls, navigation bars, sheets, dialogs, and theming
Navigation Compose	2.9.8	Single-activity destination graph and typed route arguments
Lifecycle and ViewModel	2.10.0	Screen state, coroutine lifecycle, and lifecycle-aware Flow collection
Room	2.8.4	Local SQLite persistence, entity mapping, DAO validation, migrations, and reactive queries
KSP	2.3.9	Room code generation at compile time

Technology		Version or configuration	Role
Kotlin and Flow	Coroutines	Via AndroidX dependencies	Asynchronous database work, network work, timers, and observable state
Coil		2.7.0	Loading exercise images and animated GIF demonstrations
OkHttp		4.12.0	HTTPS client for the Groq chat-completions request
Markdown Renderer		0.45.0	Material 3 rendering of formatted AI replies in Compose
Compose	Reorderable	0.9.6	Drag-and-drop order editing for plan exercises
Android services	platform	AlarmManager, notifications, TTS, MediaPlayer	Daily reminders, boot restore, voice preview, and screen behavior

Jetpack Compose is a declarative toolkit in which composable functions describe UI from state [5]. Room represents database tables as annotated entities and validates their relationships and queries during compilation [4]. These technologies match the application's need for many stateful screens and a local-first data model.

6 Setup and Installation

6.1 Requirements

To build the project, a developer needs:

- Android Studio with a compatible JDK, preferably the bundled JDK 17 or later;
- Android SDK Platform 37 and the build tools requested by Gradle;
- an emulator or physical Android device running API 24 or later;
- internet access for the first dependency sync and for chatbot requests;
- a Groq API key if the chatbot should return live answers.

6.2 Clone and Open

```
git clone https://github.com/hafonium/HomeWorkout.git
cd HomeWorkout
```

Open the repository root in Android Studio and allow Gradle synchronization to finish. The repository contains one application module, so no additional module selection is required.

6.3 Configure Local Properties

The root `local.properties` file must contain the Android SDK path. To enable the chatbot, add a personal Groq token:

```
sdk.dir=C:\\path\\to\\Android\\Sdk
GROQ_API_KEY=your_personal_groq_key
```

`local.properties` is ignored by Git and must never be committed. Without `GROQ_API_KEY`, the main local features can still build, but live chat requests will return the stored fallback error message.

6.4 Build from the Command Line

On Windows:

```
.\gradlew.bat assembleDebug
```

On macOS or Linux:

```
./gradlew assembleDebug
```

The debug APK is generated under:

```
app/build/outputs/apk/debug/app-debug.apk
```

Install it with Android Studio's Run action or Android Debug Bridge:

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

6.5 Runtime Permissions

The application declares internet, notification, boot-completed, exact-alarm, and battery-optimization permissions. On Android 13 or later, grant notification permission if reminders are needed. Depending on the Android version and device manufacturer, the user may also need to permit exact alarms and remove battery restrictions.

Exercise records and plan templates are bundled in the APK and seeded on first launch. The first database creation may take longer than later launches because hundreds of exercises and related rows are inserted.

6.6 Important Branch Setup

The current repository has separate `main` and `chatbot` development lines. For a clean chatbot-only test, a fresh installation can create the chatbot schema. Switching an installed app between the latest main build and the chatbot build is unsafe because both declare database version 6 with different schemas.

The correct development setup is to merge the two lines and add a version 6-to-7 Room migration before testing upgrades. Clearing application data can bypass the identity mismatch for a temporary demonstration, but it deletes local plans, settings, progress, weight records, and conversations and is not an acceptable release solution.

6.7 Verification Commands

The normal verification set is:

```
. \gradlew.bat testDebugUnitTest  
. \gradlew.bat lintDebug  
. \gradlew.bat assembleDebug
```

The current chatbot checkout assembles successfully. Its unit-test task reports `NO-SOURCE`, because test sources are not present on that branch. Lint currently stops on one API-level error: `android:windowLightNavigationBar` is declared in the base values resource even though it requires API 27 and the application supports API 24. The resource should be moved to a `values-v27` file or guarded with an appropriate resource qualifier.

7 Work Division

The following division is based on the member list and observable Git history. Each row lists the primary responsibility rather than claiming that features were developed without review or integration by other members.

7.1 Shared Responsibilities

All members contributed to integration, UI review, manual testing, and conflict resolution as features crossed module boundaries. The project uses shared architecture rules and common UI components so that work from different feature branches can be connected through the same repository and use-case interfaces. Final responsibilities still include merging the history/weight and chatbot development lines, resolving the Room schema version, and completing an end-to-end regression pass.

Table 5: Team work division

Member	Student ID	Primary contributions
Nguyễn Gia Hào	24125008	Initial project setup and architecture; Room and data-layer direction; Settings screens and persistence; notifications, exact-alarm scheduling, boot restoration, and related debugging; exercise asset cleanup; branch integration and pull-request management.
Cao Quảng Hưng	24125010	Weight-tracking screen and dashboard components; weight repository and domain calculations; database-backed workout history, calendar, weekly summary, and session snapshot migration; migration and history tests; implementation analysis documents.
Huỳnh Chí Tôn	24125020	Main Compose UI and navigation development; workout plan presentation and player updates; custom workout-plan flow; streak and report UI revisions; floating AI chatbot, conversation UI, Groq integration, and chat persistence.
Bùi Nguyễn Đức Trọng	24125021	Exercise search and filter flow; add and replace exercise screens; plan exercise editing and drag reordering; achievements and badge evaluation; custom-plan management; conflict resolution and integration for plan-management work.

8 Evaluation and Self-Assessment

8.1 Assessment Method

The implementation was assessed through repository inspection, Gradle tasks, existing tests, and a runtime reproduction on a connected Android device. Claims in this report were checked against the current Kotlin source, Room entities, bundled assets, documentation, and Git history.

The following commands were run on the chatbot checkout:

```
.\gradlew.bat testDebugUnitTest lintDebug assembleDebug
adb logcat -d -v time "AndroidRuntime:E" "*:S"
```

The compile and APK assembly tasks completed. The unit-test task had no source files on this branch. Lint found one error and 59 warnings. Runtime logging identified a Room identity-hash

failure when opening a database previously created by the main branch.

8.2 What Was Completed

Table 6: Completed project outcomes

Outcome	Assessment
Functional training workflow	Users can select a plan, open a specific day, inspect exercises, follow the player, and complete or abandon a persisted session.
Meaningful personalization	Profile answers affect template ranking and exercise selection; plan contents remain editable afterward.
Large local content base	771 exercises and 16 templates provide enough variation for filters, recommendations, and custom plans.
Maintainable feature boundaries	UI, domain, and data responsibilities are separated. ViewModels use focused use cases instead of DAOs.
Local-first behavior	Core exercise, plan, settings, progress, and achievement flows do not require a server after installation.
Plan ownership and editing	System and custom plans share the main flows, while deletion is limited to user-created content.
Motivation features	Weekly goals, streaks, ten badge definitions, unlock dialogs, reminders, and progress displays are implemented.
Expanded reporting	The main branch contains database-backed history, session snapshots, weight trends, and BMI calculations.
AI prototype	The chatbot branch contains a complete UI-to-network path with saved sessions, a rolling summary, structured output, and fallback handling.

8.3 Strengths

The strongest part of the application is the continuity between features. The exercise catalog is not an isolated list: it supports recommendation, plan detail, plan editing, custom plans, and the workout player. Completed sessions are not isolated rows: they drive weekly goals, streaks, achievements, history, and summary statistics. Settings are persisted and snapshot into sessions. This reuse gives the project more depth than a collection of unrelated screens.

The second strength is the explicit architecture. Although the project is contained in one Android module, responsibilities are divided by package and type. Domain models protect the UI from Room entities. Repository interfaces separate behavior from storage. The approach aligns with Android’s recommended layered architecture and unidirectional state flow [3].

The third strength is the attempt to handle real Android platform behavior. Reminder code

considers notification permission, exact-alarm access, reboot restoration, Doze, and vendor battery management. TTS code handles unavailable or duplicate voices. Image code supports both static media and GIFs. These details move the project beyond a static classroom mock-up.

8.4 Known Gaps and Risks

8.4.1 Branch and Database Integration

The highest-priority issue is the incompatible use of Room database version 6. Main uses version 6 for workout-session history snapshots. Chatbot uses version 6 for chat tables. When a database from one build is opened by the other, Room reports:

```
Room cannot verify the data integrity.  
Expected identity hash ... found ...
```

This prevents the app from opening in that upgrade path. It is a release blocker, not a cosmetic problem. The schema must move to version 7 after merging.

8.4.2 Automated Tests

The chatbot branch contains no active unit-test source directory and has commented test dependencies. Main contains a migration test and a history use-case test, but coverage remains narrow. Android architecture guidance recommends tests for ViewModels, data sources, and navigation when the application is more than a trivial example [3].

Priority tests should cover recommendation scoring, exercise fallback rules, next-day resolution, custom-plan transactions, badge thresholds, streak date boundaries, settings persistence, chat JSON parsing, HTTP error handling, and database migrations.

8.4.3 Static and Partially Connected UI

The current chatbot branch still has static report totals and sample history. Main implements the real versions, but they are not integrated with chat. Timer settings are saved and included in session snapshots, while the player still uses fixed values of 6 seconds for preparation and 20 seconds for rest. Music and sound controls are stored but do not control a complete playback system. The temporary workout-settings sheet also keeps some state only in memory.

8.4.4 AI Scope and Safety

The assistant is reachable and can remember a summarized conversation, but it is not yet context-aware. It does not know which plan or exercise is open unless the user types that information. The system prompt gives basic topic boundaries but does not provide a structured

risk level, deterministic warning for emergency symptoms, or approved exercise substitutions from the local catalog.

The API token is recoverable from the APK because it is compiled into `BuildConfig`. The Groq free tier is appropriate for a demonstration but has quotas and no application-controlled service guarantee. The model name is also pinned in code and may require replacement if provider support changes.

8.4.5 Platform and Quality Checks

Lint currently reports one blocking compatibility error because a navigation-bar theme attribute that requires API 27 is placed in resources used by API 24 devices. The 59 warnings should be triaged rather than hidden behind a baseline. Exact-alarm and battery-optimization permissions also require careful review before Play Store distribution.

8.5 Overall Self-Assessment

Table 7: Self-assessment by criterion

Criterion	Rating	Reason
Feature breadth	Strong	The repository covers planning, execution, personalization, progress, settings, reminders, achievements, weight, history, and AI chat.
Core workflow	Strong	The plan-to-session path is implemented with persistent state and clear navigation.
Architecture	Strong	Layer boundaries, repositories, use cases, state flows, and shared UI components are consistently present.
UI completeness	Good	Most screens are functional and styled; a few settings and report elements remain partially connected on the current branch.
Data integrity	Needs work	Migrations exist, but the independent version-6 schemas create a verified startup failure after branch switching.
Testing	Needs work	Assembly succeeds, but the current branch has no unit-test sources and lint does not pass.
Security	Prototype level	Local data is simple, but the AI key should not ship inside a public APK.
Release readiness	Not ready	Branch integration, migration, lint, regression tests, and API-key protection remain required.

The project is therefore assessed as a broad and technically meaningful prototype with a mostly complete user experience, but not yet a production-ready release. The remaining issues are identifiable and concentrated. Resolving the database integration, reconnecting configuration values, restoring tests, and protecting the AI service would substantially improve reliability without requiring a redesign of the main architecture.

8.6 Recommended Next Steps

1. Merge `origin/main` into the chatbot development line and introduce Room version 7 with a tested migration.
2. Move the API credential behind a small backend proxy and add rate limiting.
3. Pass active exercise, plan-day, profile, and recent-session context to the assistant; require user confirmation before applying suggested plan changes.
4. Connect preparation and rest settings to the player and either implement or remove incomplete audio controls.
5. Fix the API-27 theme resource, review remaining lint warnings, and make `lintDebug` pass.
6. Restore test dependencies and add unit, migration, repository, ViewModel, and navigation coverage.
7. Run clean-install and upgrade tests on API 24, 27, 33, and the current target version.

9 Conclusion

HomeWorkout demonstrates a complete direction for a local-first fitness application. It connects profile-based plan selection, a substantial exercise catalog, editable plans, a guided player, session persistence, progress features, settings, reminders, voice support, weight and history reporting, and an AI assistant through one Android codebase.

The application is strongest where features share the same data and domain rules. The same exercise records support recommendation, browsing, editing, details, and workout playback. The same completed-session records support goals, streaks, achievements, and history. Clean layer boundaries make these relationships understandable and provide a practical base for further development.

The project has not reached release quality. The chatbot and main development lines must be integrated through a new Room migration, lint must pass, test coverage must be expanded, and the AI key must be moved out of the APK. These are important issues, but they are bounded engineering tasks rather than missing product foundations. After they are addressed,

the repository can support a stable demonstration and a credible next iteration focused on context-aware fitness guidance.

10 AI Usage Statement

AI tools were used as development assistants for planning, code generation, debugging, documentation, and report drafting. Team members remained responsible for project decisions, source control, testing, and the final submitted work.

For implementation, generated suggestions were reviewed against the existing architecture and edited before inclusion. Repository code, Room entities, Gradle configuration, build output, device logs, and Git history were treated as the source of truth. The team did not assume that generated code was correct because it compiled.

For this report, AI assisted with organizing the required topics and converting repository evidence into clear prose and tables. Claims were checked against the inspected source files and actual verification results. The report explicitly records incomplete work, including the Room schema collision, missing tests on the chatbot branch, lint failure, fixed player timers, and the API-key risk.

The application's own chatbot also uses an AI model. User messages and a rolling context summary are sent to Groq, while message history is stored locally in Room. The current assistant should be treated as general informational support, not medical diagnosis or a substitute for a qualified trainer or healthcare professional.

References

- [1] Groq. Structured outputs. <https://console.groq.com/docs/structured-outputs>. Accessed September 4, 2026.
- [2] Android Developers. Guide to app architecture. <https://developer.android.com/topic/architecture>. Accessed September 4, 2026.
- [3] Android Developers. Recommendations for android architecture. <https://developer.android.com/topic/architecture/recommendations>. Accessed September 4, 2026.
- [4] Android Developers. Define data using room entities. <https://developer.android.com/training/data-storage/room/defining-data>. Accessed September 4, 2026.
- [5] Android Developers. Jetpack compose tutorial. <https://developer.android.com/develop/ui/compose/tutorial>. Accessed September 4, 2026.